

Homework #3 Solution: STA5021

통계학과 신현수 2025712931

2026-05-01

1. (10 points) The Matérn covariance function is defined by

$$C(t, s) = \frac{\sigma^2}{\Gamma(\nu) 2^{\nu-1}} \left(\frac{\sqrt{2\nu} |t - s|}{\rho} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} |t - s|}{\rho} \right), \quad \nu > 0,$$

where σ^2 is the variance parameter, ν the smoothness parameter, and ρ the range parameter. $\Gamma(\cdot)$ is the gamma function and $K_\nu(\cdot)$ is the modified Bessel function of the second kind. Let $C(t, t) = \sigma^2$.

- (a) Simulate and plot 100 i.i.d. functions f_i s on a grid with 50 evenly spaced points $0 = t_1 < \dots < t_{50} = 100$, such that $(f_1(t_1), \dots, f_1(t_{50}))^\top \sim N_{50}(\mathbf{0}_{50}, \mathbf{C})$ with $\mathbf{C} = (C(t_i, t_j))_{i,j} \in \mathbb{R}^{50 \times 50}$. Set $\nu = 1/2$, $\sigma^2 = 1$, and $\rho = 1$. Use `gamma()`, `besselK()`, and `mvtnorm::rmvnorm()` functions in R.

[Solution]

Matérn 공분산 함수는 다음과 같이 정의된다.

$$C(t, s) = \frac{\sigma^2}{\Gamma(\nu) 2^{\nu-1}} \left(\frac{\sqrt{2\nu} |t - s|}{\rho} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} |t - s|}{\rho} \right), \quad \nu > 0,$$

단, $C(t, t) = \sigma^2$ 입니다. $\nu = 1/2$, $\sigma^2 = 1$, $\rho = 1$ 로 설정하고, $0 \leq t \leq 100$ 범위의 50개 등간격 grid 위에서 100개의 함수를 생성한다.

```
set.seed(5021)

# grid
t.grid <- seq(0, 100, length.out = 50)
n.curves <- 100

# Matérn covariance
matern_cov <- function(h, nu = 1/2, sigma2 = 1, rho = 1) {
  out <- numeric(length(h))
  idx0 <- (h == 0)
  out[idx0] <- sigma2 # C(t, t) = sigma^2

  z <- sqrt(2 * nu) * h[!idx0] / rho
  out[!idx0] <- sigma2 / (gamma(nu) * 2^(nu - 1)) *
    (z^nu) * besselK(z, nu)
```

```

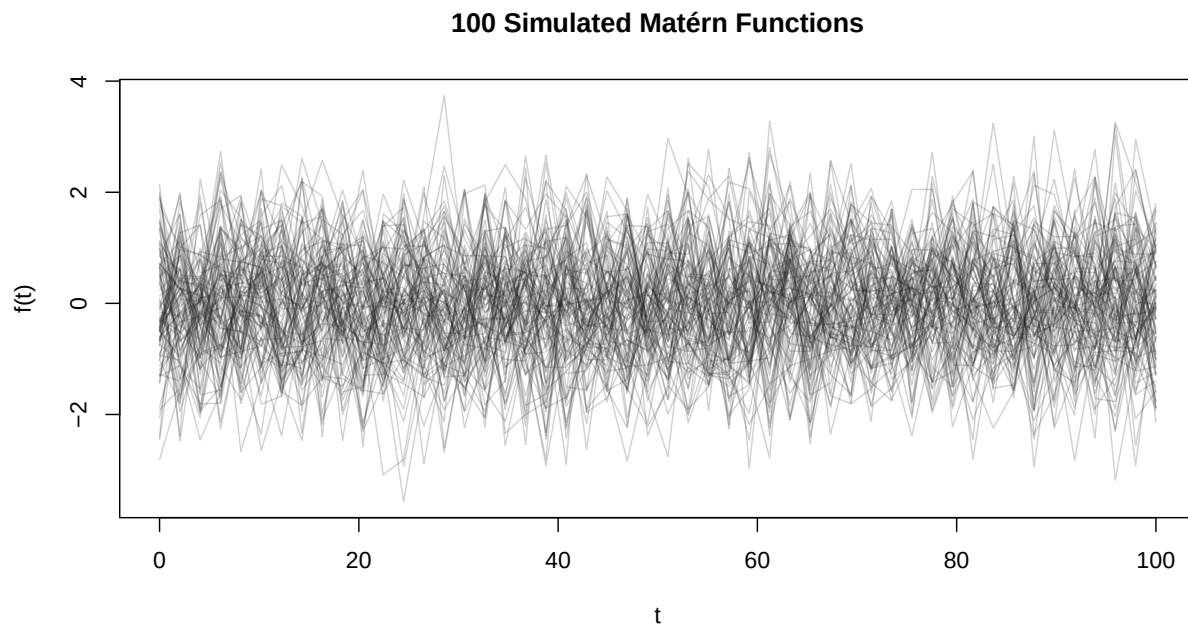
out
}

#
C.mat <- outer(t.grid, t.grid,
              function(s, t) matern_cov(abs(s - t),
                                         nu = 1/2, sigma2 = 1, rho = 1))

# 100 curves
X <- rmvnorm(n = n.curves, mean = rep(0, length(t.grid)), sigma = C.mat)

matplot(t.grid, t(X), type = "l", lty = 1, col = rgb(0.1, 0.1, 0.1, 0.2),
        xlab = "t", ylab = "f(t)", main = "100 Simulated Matérn Functions")

```



(b) Plot the first four FPCs. Do not use the `fda::pca.fd()` function; use the trapezoidal rule instead.

[Solution]

`fda::pca.fd()`를 사용하지 않고 trapezoidal rule을 사용하여 FPCA를 수행한다.

```

# Trapezoidal weights
trapw <- function(tt) {
  n <- length(tt)
  c((tt[2] - tt[1]) / 2,
    (tt[3:n] - tt[1:(n - 2)]) / 2,
    (tt[n] - tt[n - 1]) / 2)
}

w <- trapw(t.grid)
W <- diag(w)

```

```

W_half    <- diag(sqrt(w))
W_invhalf <- diag(1 / sqrt(w))

# Pointwise centering
X.center <- scale(X, center = TRUE, scale = FALSE)

#
V <- crossprod(X.center) / nrow(X.center)

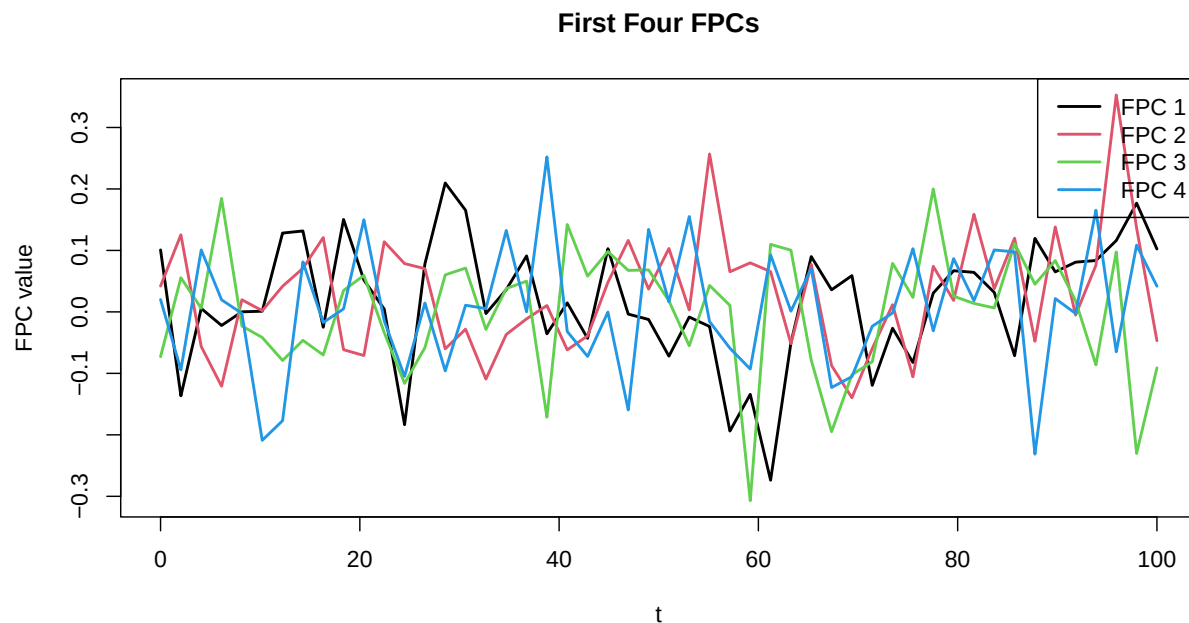
# (S = W^{1/2} V W^{1/2})
S <- W_half %*% V %*% W_half
eig <- eigen(S, symmetric = TRUE)

lambda <- pmax(eig$values, 0)
phi <- W_invhalf %*% eig$vectors[, 1:4] #

# (phi^T W phi = 1)
for (k in 1:4) {
  phi[, k] <- phi[, k] / sqrt(drop(t(phi[, k]) %*% W %*% phi[, k]))
}

matplot(t.grid, phi, type = "l", lty = 1, lwd = 2, col = 1:4,
        xlab = "t", ylab = "FPC value", main = "First Four FPCs")
legend("topright", legend = paste("FPC", 1:4),
      col = 1:4, lty = 1, lwd = 2)

```



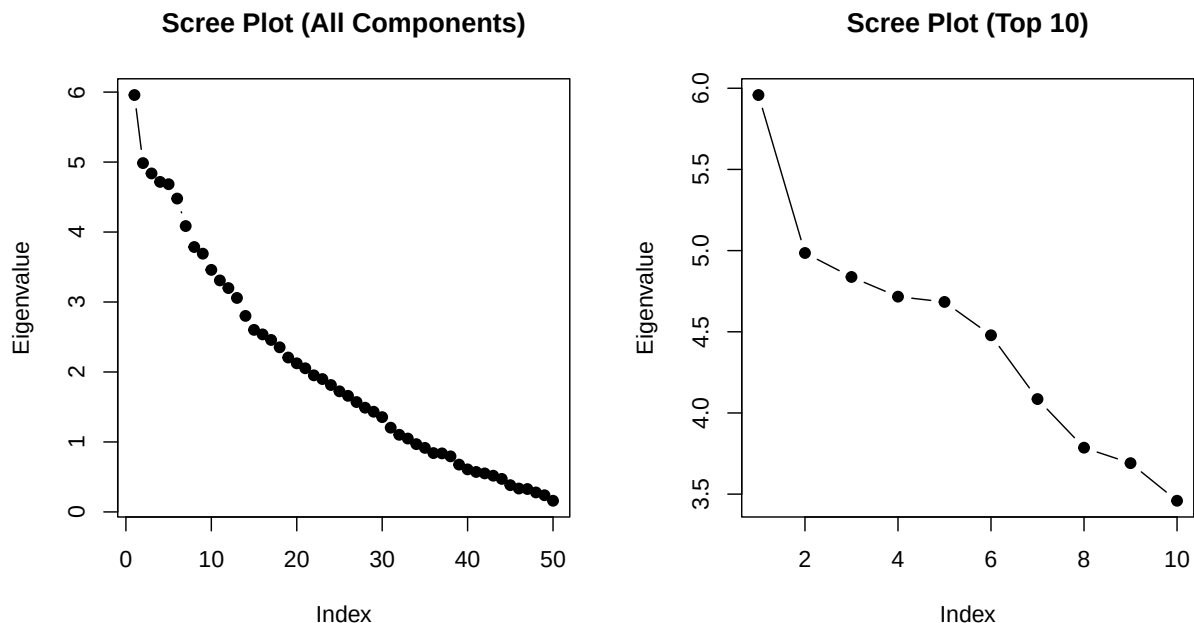
(c) Draw the scree plot.

[Solution]

```
par(mfrow = c(1, 2))

# 50 eigenvalue
plot(lambda, type = "b", pch = 19,
      xlab = "Index", ylab = "Eigenvalue",
      main = "Scree Plot (All Components)")

# 10 zoom-in
plot(1:10, lambda[1:10], type = "b", pch = 19,
      xlab = "Index", ylab = "Eigenvalue",
      main = "Scree Plot (Top 10)")
```



Interpretation.

$\nu = 1/2$ 인 Matérn covariance는 exponential covariance와 동일하다. 본 설정에서는 $\rho = 1$ 인데 도메인이 $[0, 100]$ 으로 매우 길고, grid 간격도 약 $100/49 \approx 2.04$ 이므로 인접 관측점 사이의 상관조차 $\exp(-2.04) \approx 0.13$ 정도로 크지 않다. 따라서 시뮬레이션된 함수들은 short-range dependence를 갖는 매우 rough한 sample path를 보이며, 이는 그림의 wiggly한 형태와 일치한다.

첫 4개의 FPC 역시 smooth한 저차 모드보다는 oscillatory한 형태를 보이는데, 이는 공분산 구조가 거의 대각행렬에 가까워 leading eigendirection이 특정 저주파 smooth mode에 집중되지 않고 여러 방향으로 분산되기 때문이다.

Scree plot에서도 뚜렷한 elbow 없이 eigenvalue가 완만하게 감소하여, 분산이 소수의 주성분에 집중되지 않음을 확인할 수 있다. Smooth한 covariance(예: squared exponential 또는 큰 ν)였다면 처음 몇 개 eigenvalue가 빠르게 감소하는 sharp decay가 관찰되었을 것이다.

2. (10 points) Consider the following scalar-on-function regression model with multiple functional covariates and a scalar covariate:

$$y_i = \alpha + \langle z_{1i}, \beta_1 \rangle + \langle z_{2i}, \beta_2 \rangle + x_i \gamma + \epsilon_i, \quad (i = 1, \dots, N). \quad (1)$$

To estimate the regression coefficient functions β_1 and β_2 , and two scalars α and γ , we adopt a roughness penalty approach. Given smoothing parameters $\lambda_1, \lambda_2 \geq 0$, consider the following penalized least squares criterion:

$$\sum_{i=1}^N \left(y_i - \alpha - \sum_{j=1}^2 \langle z_{ji}, \beta_j \rangle - x_i \gamma \right)^2 + \sum_{j=1}^2 \lambda_j \int [D^2 \beta_j(s)]^2 ds. \quad (2)$$

- (a) Assume that each regression coefficient function β_j is represented using a basis expansion $\beta_j = \mathbf{b}_j^\top \phi_j$, where ϕ_j is a vector of known basis functions and $\mathbf{b}_j$ is a vector of coefficients. Also, assume that each covariate function z_{ji} is represented as $\mathbf{z}_j = (z_{j1}, \dots, z_{jN})^\top = \mathbf{C}_j \psi_j$. Define appropriate matrices (e.g., $\mathbf{J}_1 = \int \psi_1(s) \phi_1^\top(s) ds$) and show that the model (1) can be written in the form of a multiple linear regression model.

[Solution]

각 β_j 를 basis 전개

$$\beta_j(s) = \phi_j(s)^\top \mathbf{b}_j$$

로, 각 covariate function을

$$z_{ji}(s) = \psi_j(s)^\top \mathbf{c}_{ji}$$

로 표현한다. 다음 행렬과 벡터들을 정의한다.

- $\mathbf{y} = (y_1, \dots, y_N)^\top \in \mathbb{R}^N$: 반응변수 벡터
- $\mathbf{x} = (x_1, \dots, x_N)^\top \in \mathbb{R}^N$: 스칼라 covariate 벡터
- $\mathbf{C}_j \in \mathbb{R}^{N \times K_{\psi_j}}$: i 번째 행이 $\mathbf{c}_{ji}^\top$ 인 행렬
- $\mathbf{J}_j = \int \psi_j(s) \phi_j(s)^\top ds \in \mathbb{R}^{K_{\psi_j} \times K_{\phi_j}}$
- $\mathbf{b}_j \in \mathbb{R}^{K_{\phi_j}}$: β_j 의 basis 계수 벡터

이때

$$\langle z_{ji}, \beta_j \rangle = \mathbf{c}_{ji}^\top \mathbf{J}_j \mathbf{b}_j, \quad (\langle z_{j1}, \beta_j \rangle, \dots, \langle z_{jN}, \beta_j \rangle)^\top = \mathbf{C}_j \mathbf{J}_j \mathbf{b}_j$$

가 성립한다. 따라서 모형 전체는

$$\mathbf{y} = \alpha \mathbf{1}_N + \mathbf{C}_1 \mathbf{J}_1 \mathbf{b}_1 + \mathbf{C}_2 \mathbf{J}_2 \mathbf{b}_2 + \mathbf{x} \gamma + \epsilon = \mathbf{Z} \theta + \epsilon$$

의 다중선형회귀 형태로 쓸 수 있으며, 여기서

$$\mathbf{Z} = [\mathbf{1}_N, \mathbf{C}_1 \mathbf{J}_1, \mathbf{C}_2 \mathbf{J}_2, \mathbf{x}] \in \mathbb{R}^{N \times (1 + K_{\phi_1} + K_{\phi_2} + 1)}, \quad \theta = (\alpha, \mathbf{b}_1^\top, \mathbf{b}_2^\top, \gamma)^\top$$

이다.

(b) Express the penalized least squares criterion (2) in a matrix form. Clearly define all matrices involved.

[Solution]

Roughness penalty matrix를

$$\mathbf{R}_j = \int D^2 \phi_j(s) D^2 \phi_j(s)^\top ds \in \mathbb{R}^{K_{\phi_j} \times K_{\phi_j}}$$

로 정의하면

$$\int [D^2 \beta_j(s)]^2 ds = \mathbf{b}_j^\top \mathbf{R}_j \mathbf{b}_j$$

이므로, penalized LS criterion은 다음과 같이 표현된다.

$$(\mathbf{y} - \mathbf{Z}\theta)^\top (\mathbf{y} - \mathbf{Z}\theta) + \theta^\top \mathbf{P}\theta$$

여기서 $\theta = (\alpha, \mathbf{b}_1^\top, \mathbf{b}_2^\top, \gamma)^\top$ 의 순서에 따라 penalty 행렬 $\mathbf{P}$ 는 다음의 블록 행렬이다.

$$\mathbf{P} = \begin{bmatrix} 0 & \mathbf{0}_{K_{\phi_1}}^\top & \mathbf{0}_{K_{\phi_2}}^\top & 0 \\ \mathbf{0}_{K_{\phi_1}} & \lambda_1 \mathbf{R}_1 & \mathbf{0}_{K_{\phi_1} \times K_{\phi_2}} & \mathbf{0}_{K_{\phi_1}} \\ \mathbf{0}_{K_{\phi_2}} & \mathbf{0}_{K_{\phi_2} \times K_{\phi_1}} & \lambda_2 \mathbf{R}_2 & \mathbf{0}_{K_{\phi_2}} \\ 0 & \mathbf{0}_{K_{\phi_1}}^\top & \mathbf{0}_{K_{\phi_2}}^\top & 0 \end{bmatrix} \in \mathbb{R}^{(1+K_{\phi_1}+K_{\phi_2}+1) \times (1+K_{\phi_1}+K_{\phi_2}+1)}$$

즉, 절편 α 와 스칼라 계수 γ 에 해당하는 행/열은 0이고, $\mathbf{b}_1, \mathbf{b}_2$ 에 해당하는 대각 블록에 각각 $\lambda_1 \mathbf{R}_1, \lambda_2 \mathbf{R}_2$ 가 위치한다.

(c) Show that the estimator of $(\alpha, \mathbf{b}_1^\top, \mathbf{b}_2^\top, \gamma)^\top$ is in the form $(\mathbf{Z}^\top \mathbf{Z} + \mathbf{P})^{-1} \mathbf{Z}^\top \mathbf{y}$. Explicitly specify the design matrix $\mathbf{Z}$ and penalty matrix $\mathbf{P}$.

[Solution]

$$(\mathbf{y} - \mathbf{Z}\theta)^\top (\mathbf{y} - \mathbf{Z}\theta) + \theta^\top \mathbf{P}\theta$$

를 θ 로 미분하면

$$-2\mathbf{Z}^\top (\mathbf{y} - \mathbf{Z}\theta) + 2\mathbf{P}\theta = 0$$

이고, 정규방정식

$$(\mathbf{Z}^\top \mathbf{Z} + \mathbf{P})\hat{\theta} = \mathbf{Z}^\top \mathbf{y}$$

로부터 추정량은 다음과 같다.

$$\hat{\theta} = (\hat{\alpha}, \hat{\mathbf{b}}_1^\top, \hat{\mathbf{b}}_2^\top, \hat{\gamma})^\top = (\mathbf{Z}^\top \mathbf{Z} + \mathbf{P})^{-1} \mathbf{Z}^\top \mathbf{y}$$

3. (30 points) In class, we introduced a permutation F -test to assess the significance of a functional covariate z in the model

$$y = \alpha + \langle z, \beta \rangle + \epsilon.$$

Conduct a simulation study to evaluate the performance of this test.

- (a) Generate simulated datasets for $z(t)$, $\beta(t)$, and ϵ under the following two scenarios:

- **Null case:** $\beta(t) = 0$ for all t .
- **Alternative case:** $\beta(t) \neq 0$.

Clearly specify your choice of $z(t)$, $\beta(t)$, and ϵ .

[Solution]

Grid. $t \in [0, 1]$ 위의 등간격 51개 점.

Functional covariate $z_i(t)$. Smooth한 predictor를 생성하기 위해 5개 Fourier basis 함수의 무작위 선형결합을 사용한다.

$$z_i(t) = a_{i1} + a_{i2} \sin(2\pi t) + a_{i3} \cos(2\pi t) + a_{i4} \sin(4\pi t) + a_{i5} \cos(4\pi t),$$

where $a_{ik} \sim N(0, \tau_k^2)$, $(\tau_1, \dots, \tau_5) = (1.0, 1.0, 0.8, 0.5, 0.5)$.

Coefficient function $\beta(t)$. 다음 시나리오를 고려한다.

- **Null:** $\beta(t) = 0$
- **Alternative (global):** $\beta(t) \propto \sin(2\pi t) + 0.5 \cos(4\pi t)$
- **Alternative (local):** $\beta(t) \propto \phi_{0.5, 0.12}(t)$ (Gaussian density)

각 alternative $\beta(t)$ 는 L_2 norm이 1이 되도록 정규화한 후 signal strength $c \in \{0.5, 1.0\}$ 를 곱하여 사용한다.

Noise. $\epsilon_i \stackrel{iid}{\sim} N(0, \sigma_e^2)$, $\sigma_e \in \{0.5, 1.0\}$.

평가 요인. 표본크기 $N \in \{50, 100\}$, noise level $\sigma_e \in \{0.5, 1.0\}$, signal strength $c \in \{0.5, 1.0\}$, $\beta(t)$ 의 형태 (global/local), 그리고 null case ($c = 0$).

```
set.seed(5021)

trapw <- function(tt) {
  n <- length(tt)
  c((tt[2] - tt[1]) / 2,
    (tt[3:n] - tt[1:(n - 2)]) / 2,
    (tt[n] - tt[n - 1]) / 2)
}

# functional predictor
simulate_z <- function(N, t.grid) {
  B <- cbind(1,
             sin(2*pi*t.grid), cos(2*pi*t.grid),
             sin(4*pi*t.grid), cos(4*pi*t.grid))
  A <- cbind(rnorm(N, 0, 1.0), rnorm(N, 0, 1.0),
             rnorm(N, 0, 0.8),
```

```

      rnorm(N, 0, 0.5), rnorm(N, 0, 0.5))
Zmat <- A %*% t(B)
scale(Zmat, center = TRUE, scale = FALSE)
}

# coefficient function beta(t)
make_beta <- function(t.grid, strength = 0, shape = c("global", "local")) {
  shape <- match.arg(shape)
  if (strength == 0) return(rep(0, length(t.grid)))
  beta <- if (shape == "global") {
    sin(2*pi*t.grid) + 0.5*cos(4*pi*t.grid)
  } else {
    dnorm(t.grid, mean = 0.5, sd = 0.12)
  }
  w <- trapw(t.grid)
  beta <- beta / sqrt(sum(beta^2 * w)) # L2-normalize
  strength * beta
}

# scalar response
simulate_y <- function(Zmat, beta, t.grid, sigma.e = 1, alpha = 0) {
  w <- trapw(t.grid)
  signal <- as.vector(Zmat %*% (beta * w))
  alpha + signal + rnorm(nrow(Zmat), 0, sigma.e)
}

# basis ( )
prep_sof <- function(t.grid, nbasis = 11) {
  bb <- create.bspline.basis(rangeval = range(t.grid),
                             nbasis = nbasis, norder = 4)
  Phi <- eval.basis(t.grid, bb)
  R <- eval.penalty(bb, Lfdobj = 2)
  w <- trapw(t.grid)
  list(Phi = Phi, R = R, w = w)
}

# fit penalized scalar-on-function regression ( basis )
fit_sof <- function(y, Zmat, prep, lambda = 1e-2) {
  Xproj <- Zmat %*% (prep$Phi * prep$w)
  Zdes <- cbind(1, Xproj)

  P <- matrix(0, ncol(Zdes), ncol(Zdes))
  P[-1, -1] <- lambda * prep$R

  theta.hat <- solve(crossprod(Zdes) + P, crossprod(Zdes, y))
  y.hat <- as.vector(Zdes %*% theta.hat)

  F.stat <- var(y.hat) / mean((y - y.hat)^2)
  list(theta.hat = theta.hat, y.hat = y.hat, F.stat = F.stat)
}

# permutation F-test (slide definition)
perm_F_test <- function(y, Zmat, prep, lambda = 1e-2, nperm = 200) {

```



```

fit0 <- fit_sof(y, Zmat, prep, lambda = lambda)
F0 <- fit0$F.stat

Fperm <- numeric(nperm)
for (b in 1:nperm) {
  y.star <- sample(y)
  Fperm[b] <- fit_sof(y.star, Zmat, prep, lambda = lambda)$F.stat
}

# slide definition: p-value = (1/M) * sum_{m=1}^M I(F_0 < F_m)
pval <- mean(F0 < Fperm)

list(F0 = F0, Fperm = Fperm, pval = pval)
}

```

- (b) Based on your simulated datasets in (a), conduct a permutation F -test and discuss how the permutation F -test's performance depends on factors such as sample size, noise level, the strength or shape of $\beta(t)$, etc.

[Solution]

다양한 표본 크기(N), 노이즈 수준(σ_e), signal strength(c), $\beta(t)$ 형태에 따른 검정 성능을 평가한다. Null case($c = 0$)는 shape에 무관하므로 한 번만 평가한다.

```

run_sim <- function(N, sigma.e, strength, shape = "global",
                    nsim = 100, nperm = 200) {
  t.grid <- seq(0, 1, length.out = 51)
  prep <- prep_sof(t.grid, nbasis = 11) #

  reject <- numeric(nsim)
  for (s in 1:nsim) {
    Zmat <- simulate_z(N, t.grid)
    beta <- make_beta(t.grid, strength = strength, shape = shape)
    y <- simulate_y(Zmat, beta, t.grid, sigma.e = sigma.e)
    test <- perm_F_test(y, Zmat, prep, nperm = nperm)
    reject[s] <- as.integer(test$pval < 0.05)
  }
  mean(reject)
}

# : null shape
res_null <- expand.grid(
  N = c(50, 100),
  sigma.e = c(0.5, 1.0),
  strength = 0,
  shape = "-",
  stringsAsFactors = FALSE
)

res_alt <- expand.grid(
  N = c(50, 100),
  sigma.e = c(0.5, 1.0),
  strength = c(0.5, 1.0),

```

```

    shape      = c("global", "local"),
    stringsAsFactors = FALSE
  )

res <- rbind(res_null, res_alt)
res$reject.rate <- NA_real_

# progress bar
pb <- txtProgressBar(min = 0, max = nrow(res), style = 3)

##      |

for (i in seq_len(nrow(res))) {
  shape_arg <- if (res$strength[i] == 0) "global" else res$shape[i]
  res$reject.rate[i] <- run_sim(
    N          = res$N[i],
    sigma.e    = res$sigma.e[i],
    strength    = res$strength[i],
    shape       = shape_arg,
    nsim        = 100,
    nperm       = 200
  )
  setTxtProgressBar(pb, i)
}

##      |=====

close(pb)

knitr::kable(res, digits = 3,
  caption = "Rejection rates of the permutation F-test")

```

Table 1: Rejection rates of the permutation F-test

N	sigma.e	strength	shape	reject.rate
50	0.5	0.0	—	0.05
100	0.5	0.0	—	0.05
50	1.0	0.0	—	0.07
100	1.0	0.0	—	0.03
50	0.5	0.5	global	0.97
100	0.5	0.5	global	1.00
50	1.0	0.5	global	0.48
100	1.0	0.5	global	0.67
50	0.5	1.0	global	1.00
100	0.5	1.0	global	1.00
50	1.0	1.0	global	0.94
100	1.0	1.0	global	1.00
50	0.5	0.5	local	0.95
100	0.5	0.5	local	1.00
50	1.0	0.5	local	0.44

N	sigma.e	strength	shape	reject.rate
100	1.0	0.5	local	0.88
50	0.5	1.0	local	1.00
100	0.5	1.0	local	1.00
50	1.0	1.0	local	0.96
100	1.0	1.0	local	1.00

Discussion

시뮬레이션 결과를 바탕으로 순열 F -검정(Permutation F -test)의 특성을 정리하면 다음과 같다.

먼저 size 측면에서, null case($c = 0$)의 기각률은 $0.03 \sim 0.07$ 범위로 평균 약 0.05 를 보여 nominal level $\alpha = 0.05$ 에 가까웠다. $n_{sim} = 100$ 의 Monte Carlo 표준오차가 $\sqrt{0.05 \times 0.95/100} \approx 0.022$ 임을 고려하면 이 정도 변동은 시뮬레이션 잡음 범위 안이며, size가 비교적 잘 통제되고 있음을 알 수 있다.

Power 측면에서는 표본 크기, 잡음 수준, 신호 강도가 모두 예상되는 방향으로 영향을 주었다. N 이 50에서 100으로 늘어나면 검정력이 일관되게 상승했고($c = 0.5, \sigma_e = 1.0$ 조건에서 global $0.48 \rightarrow 0.67$, local $0.44 \rightarrow 0.88$), 반대로 σ_e 가 0.5에서 1.0으로 커지면 검정력이 절반 이하로 떨어지기도 했다($N = 50, c = 0.5$, global에서 $0.97 \rightarrow 0.48$). 신호 강도 c 가 0.5에서 1.0으로 커지면 검정력은 단조 증가했으며, $c = 1.0$ 인 시나리오 대부분에서는 0.94 이상으로 거의 포화되었다.

$\beta(t)$ 의 형태에 따른 차이는 제한적이었다. $N = 100, \sigma_e = 1.0, c = 0.5$ 조건에서 local(0.88)이 global(0.67)보다 검정력이 높게 나왔지만, 다른 조건에서는 두 형태의 차이가 크지 않았다. 이는 이번 설계의 $z(t)$ 변동 구조와 local 계수 함수가 비교적 잘 맞은 결과로 보인다.

구현 측면에서, smoothing parameter λ 는 계산 비용을 고려해 10^{-2} 로 고정했다. 또한 R의 `var()` 함수는 $N - 1$ 로 나눈 표본분산을 사용하지만, permutation test가 단조 변환에 불변이므로 p -value 결과에는 영향을 주지 않는다.

종합하면, permutation F -test는 귀무가설 하에서 nominal size를 유지하고 표본 크기, 잡음 수준, 신호 강도에 따라 검정력이 예상대로 변화함을 일관되게 보였다. 다만 $\beta(t)$ 의 형태에 따른 차이는 추가 시뮬레이션을 통한 확인이 필요해 보인다.